hexens x ZHARTA

Security Review Report for Zharta

February 2026

Table of Contents

1. About Hexens

Hexens is a pioneering cybersecurity firm dedicated to establishing robust security standards for Web3 infrastructure, driving secure mass adoption through innovative protection technology and frameworks. As an industry elite experts in blockchain security, we deliver comprehensive audit solutions across specialized domains, including infrastructure security, Zero Knowledge Proof, novel cryptography, DeFi protocols, and NFTs.

Our methodology combines industry-standard security practices combined with unique methodology of two teams per audit, continuously advancing the field of Web3 security. This innovative approach has earned us recognition from industry leaders.

Since our founding in 2021, we have built an exceptional portfolio of enterprise clients, including major blockchain ecosystems and Web3 platforms.

2. Executive Summary

This report covers the security review for Zharta. This audit covered updated contracts of Zharta's peer-to-peer lending protocol for ERC20 tokens.

Our security assessment was a full review of the new code, spanning a total of 3 days.

During our review, we identified 2 High severity vulnerabilities that could have resulted in a loss of principal assets.

We also identified several minor vulnerabilities and code optimizations.

All reported issues were fixed by the development team and subsequently verified by us.

We can confidently say that the overall security and code quality have increased after completion of our audit.

3. Security Review Details

Review Led by

Kasper Zwijsen, Head of Audits

Scope

The analyzed resources are located on:

- https://github.com/Zharta/lending-erc20-protocol/blob/cd0aebc70e83a5b8d7cc04434631851c935887b1/contracts/v1/P2PLendingVaultSecuritize.vy
- https://github.com/Zharta/lending-erc20-protocol/blob/cd0aebc70e83a5b8d7cc04434631851c935887b1/contracts/v1/P2PLendingSecuritizeBase.vy
- https://github.com/Zharta/lending-erc20-protocol/blob/cd0aebc70e83a5b8d7cc04434631851c935887b1/contracts/v1/P2PLendingSecuritizeErc20.vy
- https://github.com/Zharta/lending-erc20-protocol/blob/cd0aebc70e83a5b8d7cc04434631851c935887b1/contracts/v1/P2PLendingSecuritizeLiquidation.vy
- https://github.com/Zharta/lending-erc20-protocol/blob/cd0aebc70e83a5b8d7cc04434631851c935887b1/contracts/v1/P2PLendingSecuritizeRefinance.vy
- https://github.com/Zharta/lending-erc20-protocol/blob/cd0aebc70e83a5b8d7cc04434631851c935887b1/contracts/SecuritizeProxy.vy

The issues described in this report were fixed in the following commits:

- https://github.com/Zharta/lending-erc20-protocol/tree/a51890e6050dc9537c5fe42c8be63a7d243f365b
- https://github.com/Zharta/lending-erc20-protocol/tree/44ac867748647dd1075cd28e3439252fe7036f7b
- https://github.com/Zharta/lending-erc20-protocol/tree/9cdfb8147fce53a0315a54c15871d6a38d798a8b
- https://github.com/Zharta/lending-erc20-protocol/tree/d7de0216fdf9f304db3150cbb28f36e816c7bf5e

- https://github.com/Zharta/lending-erc20-protocol/tree/f2c03b5249d78dbd5bf19808af9bcfc6f3729c29
- https://github.com/Zharta/lending-erc20-protocol/tree/1dc09099a50d839e87355edc3e1eaa290725a74c

Changelog

23 February 2026	Audit start
26 February 2026	Initial report
2 March 2026	Revision received
10 March 2026	Final report

4. Severity Structure

The vulnerability severity is calculated based on two components:

1. Impact of the vulnerability
2. Probability of the vulnerability

Impact	Probability			
	Rare	Unlikely	Likely	Very likely
Low	Low	Low	Medium	Medium
Medium	Low	Medium	Medium	High
High	Medium	Medium	High	Critical
Critical	Medium	High	Critical	Critical

- **Severity Characteristics**

Smart contract vulnerabilities can range in severity and impact, and it's important to understand their level of severity in order to prioritize their resolution. Here are the different types of severity levels of smart contract vulnerabilities:

Critical

Vulnerabilities that are highly likely to be exploited and can lead to catastrophic outcomes, such as total loss of protocol funds, unauthorized governance control, or permanent disruption of contract functionality.

High

Vulnerabilities that are likely to be exploited and can cause significant financial losses or severe operational disruptions, such as partial fund theft or temporary asset freezing.

Medium	Vulnerabilities that may be exploited under specific conditions and result in moderate harm, such as operational disruptions or limited financial impact without direct profit to the attacker.
Low	Vulnerabilities with low exploitation likelihood or minimal impact, affecting usability or efficiency but posing no significant security risk.
Informational	Issues that do not pose an immediate security risk but are relevant to best practices, code quality, or potential optimizations.

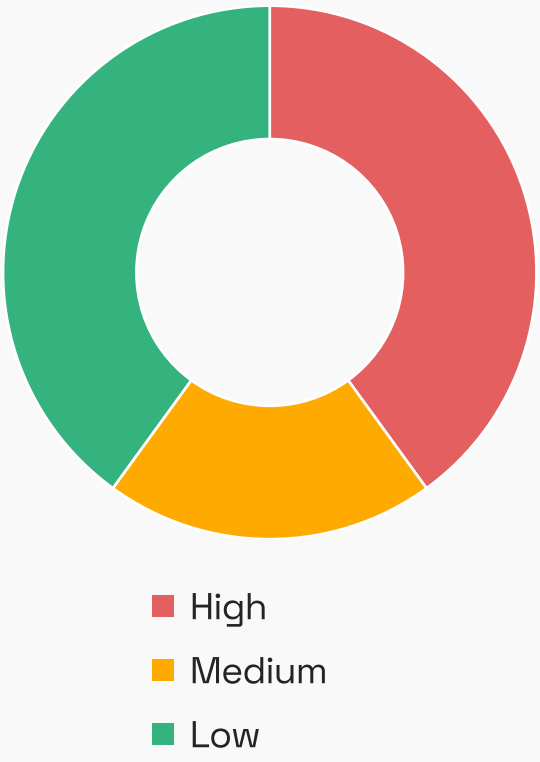
- **Issue Symbolic Codes**

Each identified and validated issue is assigned a unique symbolic code during the security research stage.

Due to the structure of the vulnerability reporting flow, some rejected issues may be missing.

5. Findings Summary

Severity	Number of findings
Critical	0
High	2
Medium	1
Low	2
Informational	0
Total:	**5**



6. Weaknesses

This section contains the list of discovered weaknesses.

ZHAR3-2 | Loan liquidation underflow could lead to frozen assets

Fixed

Severity:	Probability:	Impact:
High	Likely	High

Path:

contracts/v1/P2PLendingSecuritizeLiquidation.vy

Description:

In the **P2PLendingSecuritizeLiquidation::liquidate_loan()** function, if the loan has been redeemed, the redemption payment token will be split into **in_vault_payment_token** and **liquidation_fee**:

```
if liquidation_fee <= in_vault_payment_token:
    in_vault_payment_token -= liquidation_fee
else:
    liquidation_fee_collateral = min(in_vault_collateral, (liquidation_fee - in_vault_payment_token) *
rate.denominator * collateral_token_decimals // (rate.numerator * payment_token_decimals))
    liquidation_fee = in_vault_payment_token
    in_vault_payment_token = 0
```

However, after that, there is no check to ensure that **outstanding_debt** is greater than **in_vault_payment_token + liquidation_fee** (the total redeemed payment tokens) before calling **_received_funds**.

```
if in_vault_payment_token >= outstanding_debt:
    ...

elif in_vault_payment_token + remaining_collateral_value >= outstanding_debt:
# if remaining_collateral_value >= outstanding_debt:
    if liquidator != loan.lender:
        base._receive_funds(liquidator, outstanding_debt - in_vault_payment_token - liquidation_fee,
payment_token)
```

```
base._send_funds(loan.lender, outstanding_debt - protocol_settlement_fee_amount, payment_token)
base._send_funds(base.protocol_wallet, protocol_settlement_fee_amount, payment_token)
base._reduce_commited_liquidity(loan.lender, loan.offer_tracing_id, outstanding_debt)
```

Therefore, the **liquidate_loan()** function could revert due to an underflow in the case where **in_vault_payment_token < outstanding_debt** but **in_vault_payment_token + liquidation_fee > outstanding_debt**.

This issue can cause the loan to become permanently unliquidatable, resulting in permanently frozen funds. It could occur due to an innocent mistake by the borrower when redeeming an incorrect amount, or be exploited by a malicious borrower to prevent liquidation.

Remediation:

Separately handle the edge case so that the underflow cannot occur.

Proof of Concept :

Put the following test file into the **tests/p2p_erc20_securitize/unit** folder

- The liquidation fee ratio is already configured to 3%.
- The user redeems 50% of the collateral for 101% of the debt.
- **liquidate_loan()** will revert.

```
import boa
import pytest

from ..conftest_base import (
    ZERO_ADDRESS,
    ZERO_BYTES32,
    Offer,
    RedeemResult,
    SecuritizeLoan,
    SignedRedeemResult,
    calc_collateral_from_ltv,
    calc_ltv,
    compute_securitize_loan_hash,
    compute_signed_offer_id,
    get_last_event,
    get_securitize_loan_mutations,
    replace_namedtuple_field,
    sign_offer,
    sign_redeem_result,
)

BPS = 10000
DAY = 86400

# Empty redeem result for non-redeemed loan liquidations
EMPTY_REDEEM_RESULT = SignedRedeemResult()


@pytest.fixture(autouse=True)
def lender_funds(lender, usdc):
    usdc.mint(lender, 10**12)


@pytest.fixture(autouse=True)
def borrower_funds(borrower, usdc):
    usdc.mint(borrower, 10**12)
```

```python
@pytest.fixture
def protocol_fees(p2p_usdc_weth):
    settlement_fee = 1000
    upfront_fee = 11
    p2p_usdc_weth.set_protocol_fee(upfront_fee, settlement_fee, sender=p2p_usdc_weth.owner())
    p2p_usdc_weth.change_protocol_wallet(p2p_usdc_weth.owner(), sender=p2p_usdc_weth.owner())
    p2p_usdc_weth.set_partial_liquidation_fee(500, sender=p2p_usdc_weth.owner())
    p2p_usdc_weth.set_full_liquidation_fee(300, sender=p2p_usdc_weth.owner())
    return settlement_fee


@pytest.fixture(autouse=True)
def kyc_lender(lender, kyc_for, kyc_validator_contract):
    return kyc_for(lender, kyc_validator_contract.address)


@pytest.fixture(autouse=True)
def kyc_borrower(borrower, kyc_for, kyc_validator_contract):
    return kyc_for(borrower, kyc_validator_contract.address)


@pytest.fixture
def offer_usdc_weth(now, borrower, lender, oracle, lender_key, usdc, weth, p2p_usdc_weth):
    principal = 1000 * 10**6
    offer = Offer(
        principal=principal,
        apr=1000,
        payment_token=usdc.address,
        collateral_token=weth.address,
        duration=10 * DAY,
        origination_fee_bps=100,
        min_collateral_amount=int(0.5e18),
        max_iltv=5000,
        available_liquidity=principal,
        call_eligibility=0,
        call_window=0,
        liquidation_ltv=6000,
        oracle_addr=oracle.address,
        expiration=now + 100,
        lender=lender,
        borrower=borrower,
        tracing_id=32 * b"\1",
```

```
    )
    return sign_offer(offer, lender_key, p2p_usdc_weth.address)


@pytest.fixture
def ongoing_loan_usdc_weth(
    p2p_usdc_weth,
    offer_usdc_weth,
    usdc,
    weth,
    borrower,
    lender,
    lender_key,
    now,
    protocol_fees,
    kyc_borrower,
    kyc_lender,
    oracle,
):
    offer = offer_usdc_weth.offer
    principal = offer.principal
    collateral_amount = int(0.6e18)
    lender_approval = principal + (p2p_usdc_weth.protocol_upfront_fee() - offer.origination_fee_bps) * principal
// BPS

    weth.deposit(value=collateral_amount, sender=borrower)
    weth.approve(p2p_usdc_weth.wallet_to_vault(borrower), collateral_amount, sender=borrower)
    usdc.mint(lender, lender_approval)
    usdc.approve(p2p_usdc_weth.address, lender_approval, sender=lender)

    loan_id = p2p_usdc_weth.create_loan(
        offer_usdc_weth, principal, collateral_amount, kyc_borrower, kyc_lender, sender=borrower
    )

    loan = SecuritizeLoan(
        id=loan_id,
        offer_id=compute_signed_offer_id(offer_usdc_weth),
        offer_tracing_id=offer.tracing_id,
        initial_amount=principal,
        amount=principal,
        apr=offer.apr,
        payment_token=offer.payment_token,
        collateral_token=offer.collateral_token,
```

```
        maturity=now + offer.duration,
        start_time=now,
        accrual_start_time=now,
        borrower=borrower,
        lender=lender,
        collateral_amount=collateral_amount,
        min_collateral_amount=offer.min_collateral_amount,
        origination_fee_amount=offer.origination_fee_bps * principal // BPS,
        protocol_upfront_fee_amount=p2p_usdc_weth.protocol_upfront_fee() * principal // BPS,
        protocol_settlement_fee=p2p_usdc_weth.protocol_settlement_fee(),
        partial_liquidation_fee=p2p_usdc_weth.partial_liquidation_fee(),
        full_liquidation_fee=p2p_usdc_weth.full_liquidation_fee(),
        call_eligibility=offer.call_eligibility,
        call_window=offer.call_window,
        liquidation_ltv=offer.liquidation_ltv,
        oracle_addr=offer.oracle_addr,
        initial_ltv=offer.max_iltv,
        call_time=0,
        vault_id=0,
        redeem_start=0,
        redeem_residual_collateral=0,
    )
    assert compute_securitize_loan_hash(loan) == p2p_usdc_weth.loans(loan_id)
    return loan


@pytest.fixture
def p2p_erc20_proxy(p2p_usdc_weth, p2p_lending_erc20_proxy_contract_def):
    return p2p_lending_erc20_proxy_contract_def.deploy(p2p_usdc_weth.address)

# ============================================================================
# REDEEMED LOAN LIQUIDATION TESTS
# ============================================================================


@pytest.fixture
def redeemed_loan_with_collateral(
    p2p_usdc_weth,
    ongoing_loan_usdc_weth,
    usdc,
    weth,
    borrower,
    now,
```

```
    owner_key,
    securitize_redemption_wallet,
):
    """
    Create a redeemed loan where the vault has both payment tokens and residual collateral.
    """
    loan = ongoing_loan_usdc_weth
    residual_collateral = loan.collateral_amount // 2  # Keep 25% as collateral

    # Redeem the loan with residual collateral
    p2p_usdc_weth.redeem(loan, residual_collateral, sender=loan.borrower)
    # Update loan state to reflect redemption
    redeemed_loan = replace_namedtuple_field(
        loan,
        redeem_start=now,
        redeem_residual_collateral=residual_collateral,
    )

    # Get the vault address for this loan
    vault_addr = p2p_usdc_weth.vault_id_to_vault(borrower, loan.vault_id)

    # Partial payment from redemption (covers part of debt)
    outstanding_debt_at_maturity = loan.amount + loan.get_interest(loan.maturity)
    payment_redeemed = outstanding_debt_at_maturity * 101 // 100 # 101% of debt

    # Mint payment tokens to vault
    usdc.mint(vault_addr, payment_redeemed)


    print(loan.collateral_amount, outstanding_debt_at_maturity)


    # Create redeem result
    redeem_result = RedeemResult(
        vault=vault_addr,
        collateral_redeemed=0,  # collateral_redeemed is from redemption, not residual
        payment_redeemed=payment_redeemed,
        timestamp=now + 1,
        redeem_wallet=securitize_redemption_wallet,
    )

    return redeemed_loan, redeem_result, payment_redeemed, residual_collateral
```

```python
def test_issue_2(
    p2p_usdc_weth,
    redeemed_loan_with_collateral,
    usdc,
    weth,
    oracle,
    now,
    owner_key,
    borrower,
):
    """
    Redeemed loan with partial payment and residual collateral:
    - Liquidator pays shortfall and receives collateral
    """
    redeemed_loan, redeem_result, payment_redeemed, residual_collateral = redeemed_loan_with_collateral
    signed_redeem_result = sign_redeem_result(redeem_result, owner_key)

    liquidator = boa.env.generate_address("liquidator")

    # Make loan defaulted
    boa.env.time_travel(seconds=redeemed_loan.maturity - now + 1)

    # Calculate expected values
    outstanding_debt = redeemed_loan.amount + redeemed_loan.get_interest(redeemed_loan.maturity)
    liquidation_fee = outstanding_debt * redeemed_loan.full_liquidation_fee // BPS

    # For this fixture, liquidation_fee > in_vault_payment_token,
    # so liquidation_fee is partially taken from payment, rest from collateral
    # Let's verify this by checking balances
    liquidator_collateral_before = weth.balanceOf(liquidator)
    vault_addr = p2p_usdc_weth.vault_id_to_vault(borrower, redeemed_loan.vault_id)

    # Liquidator needs to fund the shortfall
    # The exact amount depends on the contract logic
    usdc.mint(liquidator, outstanding_debt)
    usdc.approve(p2p_usdc_weth.address, outstanding_debt, sender=liquidator)

    p2p_usdc_weth.liquidate_loan(redeemed_loan, signed_redeem_result, sender=liquidator)
```

ZHAR3-6 | Lenders are at risk of losing their funds if they liquidate their loan which was previously redeemed

Fixed

Severity:	Probability:	Impact:
High	Unlikely	Critical

Path:

contracts/v1/P2PLendingSecuritizeLiquidation.vy

Description:

In the P2PLendingSecuritizeLiquidation contract, the **liquidate_loan()** function allows the liquidation of a redeemed loan. To do this, it first calls the **withdraw_funds()** function from the vault to pull the redemption results (payment tokens) into the lending contract in order to continue the liquidation process.

```
extcall _vault.withdraw_funds(payment_token, in_vault_payment_token + liquidation_fee)
```

However, if the liquidator is the lender of the loan, the liquidation logic in the **P2PLendingSecuritizeLiquidation** contract remains similar to that of the **P2PLendingVaultedLiquidation** contract. This means that the **liquidate_loan()** function does not consider the redemption payment tokens received from the vault, causing the lender to be unable to receive those funds.

```
elif in_vault_payment_token + remaining_collateral_value >= outstanding_debt:
# if remaining_collateral_value >= outstanding_debt:
    if liquidator != loan.lender:
        base._receive_funds(liquidator, outstanding_debt - in_vault_payment_token - liquidation_fee,
payment_token)
        base._send_funds(loan.lender, outstanding_debt - protocol_settlement_fee_amount, payment_token)
        base._send_funds(base.protocol_wallet, protocol_settlement_fee_amount, payment_token)
        base._reduce_commited_liquidity(loan.lender, loan.offer_tracing_id, outstanding_debt)
    else:
        base._transfer_funds(liquidator, base.protocol_wallet, protocol_settlement_fee_amount, payment_token)
        base._send_funds(liquidator, liquidation_fee, payment_token)

    base._send_collateral(liquidator, collateral_for_debt + liquidation_fee_collateral, _vault)
    if remaining_collateral > collateral_for_debt + liquidation_fee_collateral:
        base._send_collateral(loan.borrower, remaining_collateral - collateral_for_debt - liquidation_fee_collateral,
_vault)
```

```
else:
    if liquidator != loan.lender:
        base._receive_funds(liquidator, remaining_collateral_value - in_vault_payment_token - liquidation_fee,
payment_token)
        base._send_funds(loan.lender, remaining_collateral_value - protocol_settlement_fee_amount,
payment_token)
        base._send_funds(base.protocol_wallet, protocol_settlement_fee_amount, payment_token)
        base._reduce_commited_liquidity(loan.lender, loan.offer_tracing_id, remaining_collateral_value)
    else:
        base._transfer_funds(liquidator, base.protocol_wallet, protocol_settlement_fee_amount, payment_token)

    base._send_funds(liquidator, liquidation_fee, payment_token)
    base._send_collateral(liquidator, remaining_collateral, _vault)
```

As shown in the code above, if the liquidator is the lender of the loan, the liquidation process does not send the **in_vault_payment_token** amount of funds. This causes the lender to lose their legitimate payment funds, while those funds remain in the lending contract.

Remediation:

In case of the liquidator is the lender of the loan, redeemed payment tokens (**in_vault_payment_token**) should also be sent to liquidator.

Proof of Concept :

Put the following test file into the **tests/p2p_erc20_securitize/unit** folder

- The liquidation fee ratio is configured to 0 in this test
- The user redeems 100% of the collateral for 99% of the debt. Therefore, there is no collateral remaining in the loan.
- The lender calls **liquidate_loan()**, but receives 0 payment tokens. They lose the redeemed payment tokens from the redemption (99% of the debt).

```
import boa
import pytest

from ..conftest_base import (
    ZERO_ADDRESS,
    ZERO_BYTES32,
    Offer,
    RedeemResult,
    SecuritizeLoan,
    SignedRedeemResult,
    calc_collateral_from_ltv,
    calc_ltv,
    compute_securitize_loan_hash,
    compute_signed_offer_id,
    get_last_event,
    get_securitize_loan_mutations,
    replace_namedtuple_field,
    sign_offer,
    sign_redeem_result,
)

BPS = 10000
DAY = 86400

# Empty redeem result for non-redeemed loan liquidations
EMPTY_REDEEM_RESULT = SignedRedeemResult()


@pytest.fixture(autouse=True)
def lender_funds(lender, usdc):
    usdc.mint(lender, 10**12)


@pytest.fixture(autouse=True)
```

```
def borrower_funds(borrower, usdc):
    usdc.mint(borrower, 10**12)


@pytest.fixture
def protocol_fees(p2p_usdc_weth):
    settlement_fee = 1000
    upfront_fee = 11
    p2p_usdc_weth.set_protocol_fee(upfront_fee, settlement_fee, sender=p2p_usdc_weth.owner())
    p2p_usdc_weth.change_protocol_wallet(p2p_usdc_weth.owner(), sender=p2p_usdc_weth.owner())
    p2p_usdc_weth.set_partial_liquidation_fee(500, sender=p2p_usdc_weth.owner())
    p2p_usdc_weth.set_full_liquidation_fee(0, sender=p2p_usdc_weth.owner())
    return settlement_fee


@pytest.fixture(autouse=True)
def kyc_lender(lender, kyc_for, kyc_validator_contract):
    return kyc_for(lender, kyc_validator_contract.address)


@pytest.fixture(autouse=True)
def kyc_borrower(borrower, kyc_for, kyc_validator_contract):
    return kyc_for(borrower, kyc_validator_contract.address)


@pytest.fixture
def offer_usdc_weth(now, borrower, lender, oracle, lender_key, usdc, weth, p2p_usdc_weth):
    principal = 1000 * 10**6
    offer = Offer(
        principal=principal,
        apr=1000,
        payment_token=usdc.address,
        collateral_token=weth.address,
        duration=10 * DAY,
        origination_fee_bps=100,
        min_collateral_amount=int(0.5e18),
        max_iltv=5000,
        available_liquidity=principal,
        call_eligibility=0,
        call_window=0,
        liquidation_ltv=6000,
        oracle_addr=oracle.address,
        expiration=now + 100,
```

```
        lender=lender,
        borrower=borrower,
        tracing_id=32 * b"\1",
    )
    return sign_offer(offer, lender_key, p2p_usdc_weth.address)


@pytest.fixture
def ongoing_loan_usdc_weth(
    p2p_usdc_weth,
    offer_usdc_weth,
    usdc,
    weth,
    borrower,
    lender,
    lender_key,
    now,
    protocol_fees,
    kyc_borrower,
    kyc_lender,
    oracle,
):
    offer = offer_usdc_weth.offer
    principal = offer.principal
    collateral_amount = int(0.6e18)
    lender_approval = principal + (p2p_usdc_weth.protocol_upfront_fee() - offer.origination_fee_bps) * principal
// BPS

    weth.deposit(value=collateral_amount, sender=borrower)
    weth.approve(p2p_usdc_weth.wallet_to_vault(borrower), collateral_amount, sender=borrower)
    usdc.mint(lender, lender_approval)
    usdc.approve(p2p_usdc_weth.address, lender_approval, sender=lender)

    loan_id = p2p_usdc_weth.create_loan(
        offer_usdc_weth, principal, collateral_amount, kyc_borrower, kyc_lender, sender=borrower
    )

    loan = SecuritizeLoan(
        id=loan_id,
        offer_id=compute_signed_offer_id(offer_usdc_weth),
        offer_tracing_id=offer.tracing_id,
        initial_amount=principal,
        amount=principal,
```

```
        apr=offer.apr,
        payment_token=offer.payment_token,
        collateral_token=offer.collateral_token,
        maturity=now + offer.duration,
        start_time=now,
        accrual_start_time=now,
        borrower=borrower,
        lender=lender,
        collateral_amount=collateral_amount,
        min_collateral_amount=offer.min_collateral_amount,
        origination_fee_amount=offer.origination_fee_bps * principal // BPS,
        protocol_upfront_fee_amount=p2p_usdc_weth.protocol_upfront_fee() * principal // BPS,
        protocol_settlement_fee=p2p_usdc_weth.protocol_settlement_fee(),
        partial_liquidation_fee=p2p_usdc_weth.partial_liquidation_fee(),
        full_liquidation_fee=p2p_usdc_weth.full_liquidation_fee(),
        call_eligibility=offer.call_eligibility,
        call_window=offer.call_window,
        liquidation_ltv=offer.liquidation_ltv,
        oracle_addr=offer.oracle_addr,
        initial_ltv=offer.max_iltv,
        call_time=0,
        vault_id=0,
        redeem_start=0,
        redeem_residual_collateral=0,
    )
    assert compute_securitize_loan_hash(loan) == p2p_usdc_weth.loans(loan_id)
    return loan


@pytest.fixture
def p2p_erc20_proxy(p2p_usdc_weth, p2p_lending_erc20_proxy_contract_def):
    return p2p_lending_erc20_proxy_contract_def.deploy(p2p_usdc_weth.address)

# ============================================================================
# REDEEMED LOAN LIQUIDATION TESTS
# ============================================================================


@pytest.fixture
def redeemed_loan_with_collateral(
    p2p_usdc_weth,
    ongoing_loan_usdc_weth,
    usdc,
```

```
    weth,
    borrower,
    now,
    owner_key,
    securitize_redemption_wallet,
):
    """
    Create a redeemed loan where the vault has both payment tokens and residual collateral.
    """
    loan = ongoing_loan_usdc_weth
    residual_collateral = 0  # Keep 0% as collateral

    # Redeem the loan with residual collateral
    p2p_usdc_weth.redeem(loan, residual_collateral, sender=loan.borrower)
    # Update loan state to reflect redemption
    redeemed_loan = replace_namedtuple_field(
        loan,
        redeem_start=now,
        redeem_residual_collateral=residual_collateral,
    )

    # Get the vault address for this loan
    vault_addr = p2p_usdc_weth.vault_id_to_vault(borrower, loan.vault_id)

    # Partial payment from redemption (covers part of debt)
    outstanding_debt_at_maturity = loan.amount + loan.get_interest(loan.maturity)
    payment_redeemed = outstanding_debt_at_maturity * 99 // 100 # 99% of debt

    # Mint payment tokens to vault
    usdc.mint(vault_addr, payment_redeemed)


    print(loan.collateral_amount, outstanding_debt_at_maturity)


    # Create redeem result
    redeem_result = RedeemResult(
        vault=vault_addr,
        collateral_redeemed=0,  # collateral_redeemed is from redemption, not residual
        payment_redeemed=payment_redeemed,
        timestamp=now + 1,
        redeem_wallet=securitize_redemption_wallet,
    )
```

```
    return redeemed_loan, redeem_result, payment_redeemed, residual_collateral



def test_issue_3(
    p2p_usdc_weth,
    redeemed_loan_with_collateral,
    usdc,
    weth,
    oracle,
    now,
    owner_key,
    borrower,
):
    """
    Redeemed loan with partial payment and residual collateral:
    - Liquidator pays shortfall and receives collateral
    """
    redeemed_loan, redeem_result, payment_redeemed, residual_collateral = redeemed_loan_with_collateral
    signed_redeem_result = sign_redeem_result(redeem_result, owner_key)

    #liquidator is the lender
    liquidator = redeemed_loan.lender

    # Make loan defaulted
    boa.env.time_travel(seconds=redeemed_loan.maturity - now + 1)

    # Calculate expected values
    outstanding_debt = redeemed_loan.amount + redeemed_loan.get_interest(redeemed_loan.maturity)
    liquidation_fee = outstanding_debt * redeemed_loan.full_liquidation_fee // BPS

    # For this fixture, liquidation_fee > in_vault_payment_token,
    # so liquidation_fee is partially taken from payment, rest from collateral
    # Let's verify this by checking balances
    liquidator_collateral_before = weth.balanceOf(liquidator)

    vault_addr = p2p_usdc_weth.vault_id_to_vault(borrower, redeemed_loan.vault_id)

    # Liquidator needs to fund the shortfall
    # The exact amount depends on the contract logic
    usdc.mint(liquidator, outstanding_debt)
    usdc.approve(p2p_usdc_weth.address, outstanding_debt, sender=liquidator)
```

```
liquidator_payment_before = usdc.balanceOf(liquidator)
p2p_usdc_weth.liquidate_loan(redeemed_loan, signed_redeem_result, sender=liquidator)

# Verify loan is deleted
assert p2p_usdc_weth.loans(redeemed_loan.id) == ZERO_BYTES32

# Liquidator (lender) receives 0 collateral and only 3% of payment as liquidation fee
collateral_received = weth.balanceOf(liquidator) - liquidator_collateral_before
payment_received = usdc.balanceOf(liquidator) - liquidator_payment_before

assert collateral_received == 0
assert payment_received == 0
```

ZHAR3-3 | The LTV calculation for liquidation does not exclude the redemption collateral and assets

Fixed

Severity:	Probability:	Impact:
Medium	Unlikely	High

Path:

contracts/v1/P2PLendingSecuritizeLiquidation.vy#L175-L202

Description:

The **liquidate_loan()** function allows a loan that was previously redeemed to be liquidated if its LTV is greater than **liquidation_ltv** (the liquidation threshold).

However, the LTV calculation still uses **loan.collateral_amount** as the current collateral and **loan.amount + current_interest** as the current debt, even though these values are not updated after redemption. After redemption, a portion of the collateral is converted into payment tokens at a previous rate, which may differ from the current rate.

As a result, a loan that should be healthy after redemption could still be liquidated because the calculation relies on the original collateral and debt amounts. Even if the redemption payment tokens are sufficient to fully cover the debt, the loan could still be liquidated, causing a loss to the borrower.

```
if not base._is_loan_defaulted(loan):

    current_interest = base._compute_settlement_interest(loan)
    convertion_rate: base.UInt256Rational = base._get_oracle_rate(oracle_addr, oracle_reverse)
    current_ltv: uint256 = base._compute_ltv(loan.collateral_amount, loan.amount + current_interest,
convertion_rate, payment_token_decimals, collateral_token_decimals)
    assert loan.liquidation_ltv > 0, "not defaulted, partial disabled"
    assert current_ltv >= loan.liquidation_ltv, "not defaulted, ltv lt partial"

    if not is_loan_redeemed:
        principal_written_off: uint256 = 0
        collateral_claimed: uint256 = 0
        liquidation_fee: uint256 = 0
        principal_written_off, collateral_claimed, liquidation_fee = base._compute_partial_liquidation(
            loan.collateral_amount,
            loan.amount + current_interest,
            loan.initial_ltv,
            loan.partial_liquidation_fee,
```

```
            convertion_rate,
            payment_token_decimals,
            collateral_token_decimals
        )

        assert principal_written_off >= loan.amount + current_interest, "not defaulted, partial possible"

else:
    current_interest = self._compute_liquidation_interest(loan)
```

Remediation:

The **liquidate_loan()** function should exclude redemption collateral and assets from the LTV calculation.

ZHAR3-5 | The buy function in P2PLendingVaultSecuritize may be denied due to a zero-amount transfer

Fixed

Severity:	Probability:	Impact:
Low	Rare	Medium

Path:

contracts/v1/P2PLendingVaultSecuritize.vy#L186-L212

Description:

The **P2PLendingVaultSecuritize::buy()** function pulls a **stable_coin_amount** of payment tokens from **msg.sender**, then swaps that amount for collateral tokens (DS tokens). Using the **SecuritizeSwap::swap()** function, it always swaps the full **stable_coin_amount** of payment tokens for DS tokens. Therefore, the **remaining_balance** is always equal to the **initial_balance**.

```
initial_balance: uint256 = staticcall IERC20(payment_token).balanceOf(self)
extcall IERC20(payment_token).transferFrom(msg.sender, self, stable_coin_amount)
extcall IERC20(payment_token).approve(securitize_swap_contract, stable_coin_amount)
extcall SecuritizeSwap(securitize_swap_contract).swap(stable_coin_amount, min_ds_token_amount)

self.pending_transfers[self.owner] += ds_token_amount.ds_token_amount
self.pending_transfers_total += ds_token_amount.ds_token_amount

remaining_balance: uint256 = staticcall IERC20(payment_token).balanceOf(self)
extcall IERC20(payment_token).transfer(msg.sender, remaining_balance - initial_balance)
```

As a result, the transfer of **remaining_balance - initial_balance** is a zero-amount transfer. It may revert if the payment token does not allow zero-value transfers, causing the **buy()** function to be denied. Some ERC20 tokens do not allow zero-value transfers, such as BNB.

```
@external
def buy(payment_token: address, min_ds_token_amount: uint256, stable_coin_amount: uint256):
    """
    @notice Buy DS tokens using stable coins via the SecuritizeSwap contract.
    @dev Approves the SecuritizeSwap contract to spend stable coins and executes the buy operation.
    @param min_ds_token_amount The minimum amount of DS tokens to receive.
    @param stable_coin_amount The amount of stable coins to spend.
    """
```

```
    assert self._check_user(self.owner), "unauthorized"

    securitize_swap_contract: address = staticcall SecuritizeDSToken(self.token).getDSService(1<<14)

    ds_token_amount: DsTokenAmountResult = staticcall
SecuritizeSwap(securitize_swap_contract).calculateDsTokenAmount(stable_coin_amount)
    assert ds_token_amount.ds_token_amount >= min_ds_token_amount, "ds token amount lt min"

    initial_balance: uint256 = staticcall IERC20(payment_token).balanceOf(self)
    extcall IERC20(payment_token).transferFrom(msg.sender, self, stable_coin_amount)
    extcall IERC20(payment_token).approve(securitize_swap_contract, stable_coin_amount)
    extcall SecuritizeSwap(securitize_swap_contract).swap(stable_coin_amount, min_ds_token_amount)

    self.pending_transfers[self.owner] += ds_token_amount.ds_token_amount
    self.pending_transfers_total += ds_token_amount.ds_token_amount

    remaining_balance: uint256 = staticcall IERC20(payment_token).balanceOf(self)
    extcall IERC20(payment_token).transfer(msg.sender, remaining_balance - initial_balance)
```

Remediation:

The last transfer action in the **buy()** function should be removed.

ZHAR3-7 | Oracle May Rarely Return 0, Enabling Zero-Collateral Loans

Fixed

Severity:	Low	Probability:	Rare	Impact:	Medium

Path:

P2PLendingSecuritizeBase.vy#L418-L432

Description:

External price oracles such as Chainlink are designed to return positive price values under normal operation. However, due to round finalization delays, node consensus failures, feed interruptions, stale round reads, or temporary oracle outages, there exists a non-zero probability that **latestRoundData().answer** may return **0**.

Because **_get_oracle_rate()** does not validate that **answer > 0**, a rare oracle malfunction returning **0** can propagate into the LTV computation. When **oracle_reverse=True**, this results in:

- **denominator = answer = 0**
- LTV calculation evaluating to **0**
- Loan approval despite zero effective collateral value

This creates a scenario where a transient oracle anomaly can lead to fully uncollateralized loans and total lender fund loss.

```
# P2PLendingSecuritizeBase.vy:418-432
@view
@internal
def _get_oracle_rate(oracle_addr: address, oracle_reverse: bool) -> UInt256Rational:
    if oracle_reverse:
        return UInt256Rational(
            numerator=10 ** convert(staticcall AggregatorV3Interface(oracle_addr).decimals(), uint256),
            denominator=convert(
                (staticcall AggregatorV3Interface(oracle_addr).latestRoundData()).answer,
                uint256
            )
            # No validation that answer > 0
        )
    else:
        return UInt256Rational(
            numerator=convert(
                (staticcall AggregatorV3Interface(oracle_addr).latestRoundData()).answer,
```

```
            uint256
        ),
        denominator=10 ** convert(staticcall AggregatorV3Interface(oracle_addr).decimals(), uint256)
    )
```

Scenario

Although oracle failures returning 0 are rare, they are not impossible in distributed oracle systems. If such an event occurs while:

- oracle_reverse=True
- A borrower initiates a loan
- LTV validation is performed

Then:

- answer = 0
- LTV = 0
- Loan is approved unconditionally

The borrower receives funds while posting effectively worthless collateral.

Remediation:

Add strict validation before using oracle data:

```
assert answer > 0, "Invalid oracle price"
```


hexens
x
ZHARTA